

Informe Técnico — Weddy Wedding Planner

Proyecto 3 — Diseño y Arquitectura de Software

Curso: Diseño y Arquitectura de Software

Profesor: César Augusto Vega Fernández

Integrantes: Daniel Riveros y Manuel Castillo

Repositorio: <https://github.com/manuel-0310/software-proyecto>

Tabla de contenido

1. Descripción del sistema
 2. Arquitectura del sistema
 3. Modelos arquitectónicos — C4 y 4+1
 4. Desarrollo del software
 5. Pruebas automatizadas
 6. Pipeline CI/CD
 7. DevSecOps
 8. Análisis de calidad de código
 9. Observabilidad
 10. Resiliencia
 11. Decisiones de seguridad y escalabilidad
 12. Retos técnicos y soluciones
-

1. Descripción del sistema

Weddy es una plataforma de gestión de bodas que permite centralizar la administración de invitados y presupuesto en un solo lugar.

Problema que resuelve

La planificación de una boda involucra múltiples variables interdependientes. Cuando un invitado cambia su estado de asistencia, el presupuesto debe recalcularse en tiempo real. Gestionar esto manualmente con hojas de cálculo genera errores y estrés operativo.

Weddy automatiza esta dependencia: al confirmar un invitado, el sistema recalcula el presupuesto sin intervención manual.

Atributos de calidad priorizados

Atributo	Descripción
Mantenibilidad	Separación estricta entre UI, lógica de negocio y datos (Clean Architecture)
Resiliencia	La app Flutter sigue funcionando aunque el backend falle (Circuit Breaker + caché)
Observabilidad	Monitoreo en tiempo real con Prometheus, Grafana, Jaeger y Loki
Seguridad	Acceso controlado mediante autenticación JWT, análisis SAST/DAST en CI

2. Arquitectura del sistema

2.1 Arquitectura inicial — Corte 1

El sistema original era una aplicación Flutter monolítica. Toda la lógica — modelos, repositorios, servicios y UI— convivía en un solo proceso sin comunicación externa. Los datos vivían en memoria RAM y se perdían al cerrar la app.

Estructura de capas:

```
UI → Controllers → Services → Repositories → Datos en memoria
```

Patrones de diseño implementados:

- **Factory Method:** instanciación dinámica de proveedores (DJ, Catering, Fotografía) sin acoplar el controlador a clases concretas
- **Observer:** `PresupuestoObserver` y `NotificacionObserver` reaccionan automáticamente cuando un invitado cambia de estado
- **Repository:** `IInvitadoRepository` define el contrato de acceso a datos; `InvitadoRepositoryImpl` lo implementa con listas en memoria

Limitaciones identificadas:

- Imposible compartir datos entre dispositivos
- Sin capacidad de monitoreo ni auditoría
- Sin control de acceso
- Toda la lógica de negocio expuesta en el cliente

2.2 Arquitectura evolucionada — Corte 2 y 3

El sistema evolucionó hacia una **arquitectura orientada a servicios**. La lógica de negocio se extrajo a un backend independiente (FastAPI), y la app Flutter pasó a ser un cliente que consume ese backend mediante HTTP REST.

Componentes del sistema:

Componente	Tecnología	Rol	Puerto
Weddy API	FastAPI + Python 3.11	Servicio Proveedor	8000
App Weddy	Flutter (Dart)	Servicio Consumidor	—
Prometheus	Docker	Recolección de métricas	9090
Grafana	Docker	Visualización de métricas	3000
Jaeger	Docker	Trazabilidad distribuida	16686
Loki	Docker	Centralización de logs	3100
Promtail	Docker	Agente de recolección de logs	—

Estructura del repositorio:

```
/
├─ client/           → App Flutter (Servicio Consumidor)
├─ services/         → Backend FastAPI (Servicio Proveedor)
├─ tests/
│   ├─ api/          → Colección Postman/Newman
│   ├─ load/          → Pruebas de carga k6
│   └─ gui/           → Pruebas E2E Cypress
├─ monitoring/       → Configuración Prometheus, Grafana, Loki, Promtail
├─ k8s/              → Manifiestos Kubernetes
├─ docs/             → Documentación técnica
└─ .github/workflows/ → Pipelines CI/CD
```

Endpoints disponibles en la API:

Endpoint	Método	Acceso	Descripción
<code>/auth/login</code>	POST	Público	Genera token JWT
<code>/invitados/</code>	GET	Público	Lista todos los invitados
<code>/invitados/</code>	POST	Protegido	Crea un nuevo invitado
<code>/invitados/{id}/estado</code>	PATCH	Protegido	Cambia el estado de asistencia
<code>/invitados/{id}</code>	DELETE	Protegido	Elimina un invitado
<code>/presupuesto/</code>	GET	Público	Obtiene el resumen de presupuesto
<code>/presupuesto/configurar</code>	POST	Protegido	Actualiza la configuración
<code>/metrics</code>	GET	Público	Métricas para Prometheus
<code>/health</code>	GET	Público	Estado del servicio

Comparativa de evolución:

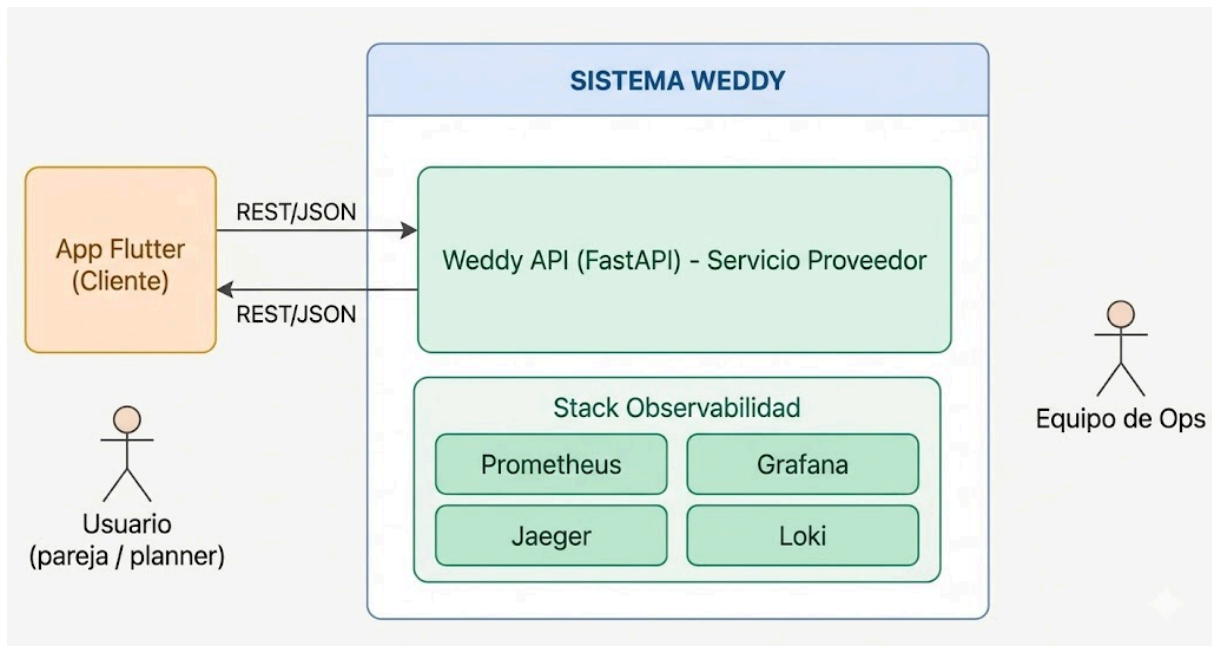
Aspecto	Corte 1	Corte 2/3
Datos	Solo en memoria del cliente	En backend centralizado
Comunicación	Ninguna	REST + JSON
Resiliencia	Ninguna	Circuit Breaker + caché
Seguridad	Ninguna	JWT (HS256) + bcrypt
Observabilidad	Ninguna	Prometheus + Grafana + Jaeger + Loki
Escalabilidad	No escalable	Backend independiente + K8s
CI/CD	Ninguno	9 jobs CI + pipeline CD

3. Modelos arquitectónicos — C4 y 4+1

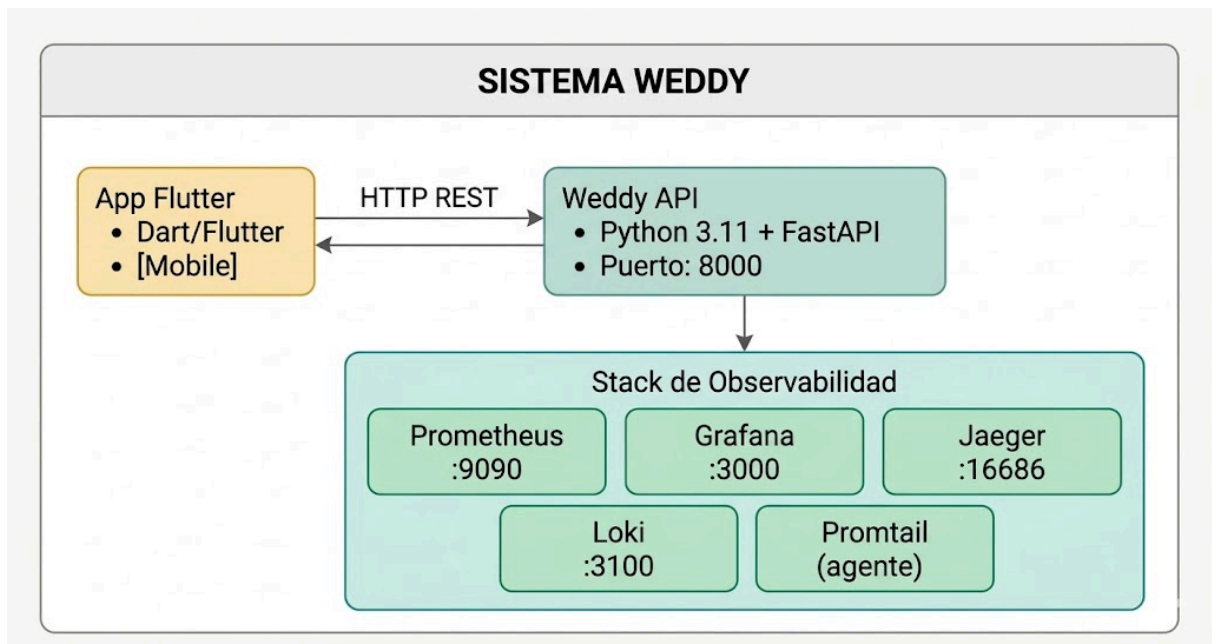
3.1 Modelo C4

Nivel 1 — Diagrama de Contexto

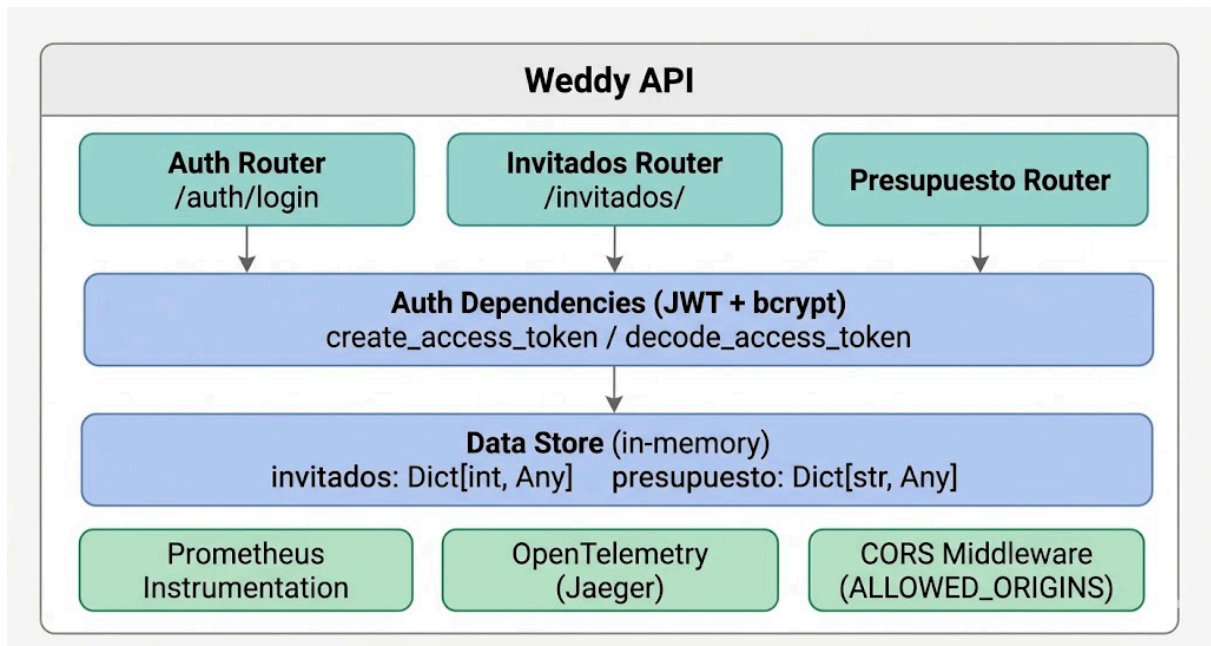
Muestra los actores externos y cómo se relacionan con el sistema.



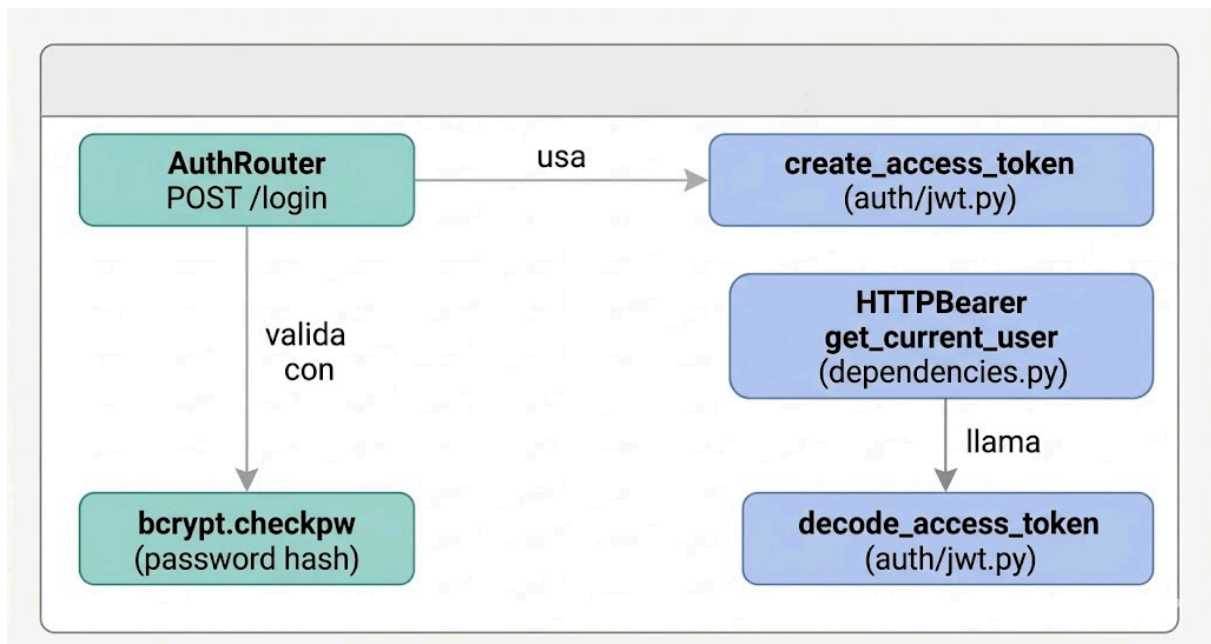
Nivel 2 — Diagrama de Contenedores



Nivel 3 — Diagrama de Componentes (Weddy API)



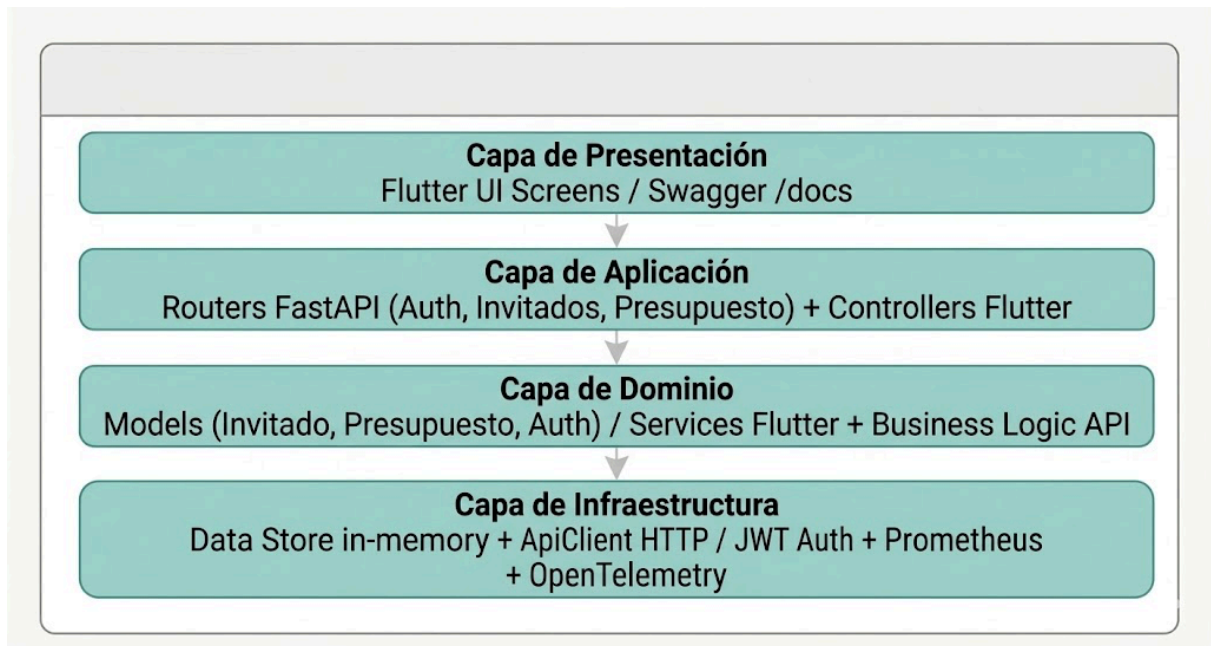
Nivel 4 — Diagrama de Código (Módulo Auth)



3.2 Vista 4+1

Vista Lógica — ¿Qué hace el sistema?

Organiza la funcionalidad en módulos de responsabilidad única.



Paquetes principales:

- `services/routers/` — controladores REST (Auth, Invitados, Presupuesto)
- `services/auth/` — autenticación JWT y dependencias
- `services/models/` — modelos de dominio con validación Pydantic
- `services/data/` — almacenamiento en memoria
- `client/lib/` — cliente Flutter organizado en capas (Clean Architecture)

Vista de Proceso — ¿Cómo fluyen los datos?

```
Proceso 1: uvicorn (servidor ASGI)
├─ Request llega → FastAPI procesa → Response
├─ Prometheus Instrumentator registra métricas por endpoint
└─ OpenTelemetry genera span → envía a Jaeger (OTLP HTTP :4318)

Proceso 2: Prometheus (scraping cada 10s)
├─ GET /metrics → almacena serie de tiempo

Proceso 3: Grafana
├─ Consulta Prometheus → renderiza dashboards en tiempo real

Proceso 4: Promtail
├─ Lee logs de contenedores Docker → envía a Loki

Proceso 5: Flutter app
├─ Circuit Breaker → ApiClient → HTTP → Weddy API
```

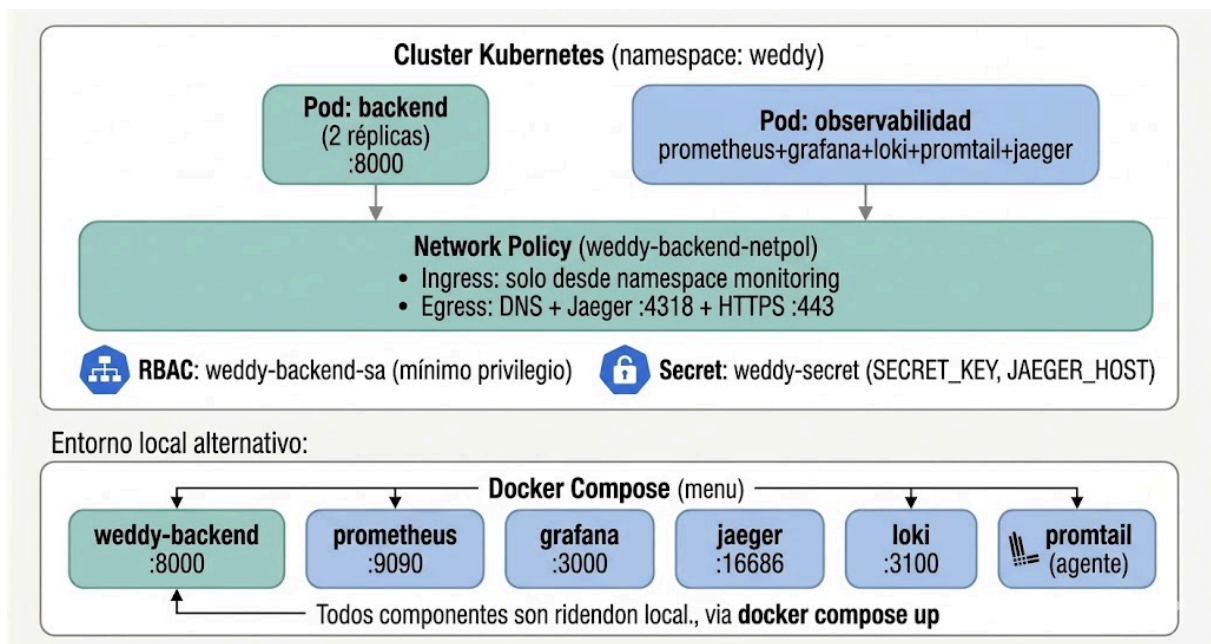
Vista de Desarrollo — ¿Cómo está organizado el código?

```

software-proyecto/
├── client/                                # App Flutter (Servicio Consumidor)
│   ├── lib/
│   │   ├── controllers/                 # Capa de presentación
│   │   ├── services/                   # Lógica de negocio
│   │   ├── repositories/               # Abstracción de datos
│   │   ├── data/                       # Datasources local y remoto
│   │   ├── models/                     # Entidades de dominio
│   │   └── patterns/                   # Circuit Breaker, Factory, Observer
│   └── test/                            # Tests Flutter unitarios
├── services/                             # Backend FastAPI (Servicio Proveedor)
│   ├── routers/                         # Endpoints REST
│   ├── auth/                           # JWT y dependencias de autenticación
│   ├── models/                         # Schemas Pydantic
│   ├── data/                           # Store en memoria
│   ├── telemetry/                      # OpenTelemetry
│   └── tests/                           # PyTest – unitarios + mocks
├── tests/
│   ├── api/                            # Colección Postman/Newman
│   ├── load/                           # k6 load test
│   └── gui/                             # Cypress E2E tests
├── monitoring/                          # Prometheus, Grafana, Loki, Promtail
├── k8s/                                 # Manifiestos Kubernetes
└── .github/workflows/                   # Pipelines CI/CD

```

Vista Física — ¿Dónde se despliega?



Vista de Escenarios — ¿Cómo se usa el sistema?

Los escenarios validan que las cuatro vistas anteriores satisfacen los requisitos del sistema.

Escenario	Actor	Flujo resumido	Vistas involucradas
UC-1: Login	Usuario	App → POST /auth/login → JWT	Lógica, Proceso
UC-2: Registrar invitado	Usuario autenticado	App + Bearer → POST /invitados/	Lógica, Proceso
UC-3: Consultar presupuesto	Usuario	App → GET /presupuesto/ → cálculo en tiempo real	Lógica
UC-4: Backend caído	Sistema	Circuit Breaker abre → fallback a caché local	Proceso
UC-5: Monitoreo	DevOps	Prometheus → Grafana dashboards	Física
UC-6: Logs centralizados	DevOps	Promtail → Loki → Grafana Explore	Física
UC-7: Trazabilidad	DevOps	OpenTelemetry → Jaeger UI por request	Proceso
UC-8: CI/CD	Developer	git push → GitHub Actions → DockerHub	Desarrollo

4. Desarrollo del software

4.1 Principios SOLID aplicados

Principio	Aplicación en el proyecto
SRP	Cada router maneja un único recurso (auth, invitados, presupuesto); cada datasource Flutter tiene una sola responsabilidad
OCP	Nuevos tipos de proveedor (Catering, DJ, Fotografía) se agregan sin modificar el Factory existente
LSP	<code>InvitadoRepositoryImpl</code> implementa <code>IInvitadoRepository</code> sin alterar el comportamiento esperado por los servicios
ISP	Los repositorios e interfaces están segregados por funcionalidad — invitados y presupuesto son contratos independientes
DIP	Los servicios Flutter dependen de interfaces (<code>IInvitadoRepository</code>), no de implementaciones concretas

4.2 Patrones de diseño implementados

Factory Method — `client/lib/patterns/factory/`

Instancia proveedores (DJ, Catering, Fotografía) sin que el controlador conozca las clases concretas. Agregar un nuevo proveedor solo requiere crear una subclase, sin tocar código existente.

Observer — `client/lib/patterns/observer/`

`PresupuestoObserver` y `NotificacionObserver` se suscriben a cambios en el estado de invitados. Cuando un invitado se confirma, el presupuesto se recalcula automáticamente sin que el controlador lo solicite explícitamente.

Repository — `client/lib/repositories/`

`IInvitadoRepository` define el contrato de acceso a datos. La implementación concreta puede intercambiarse entre datos locales y remotos sin que la lógica de negocio lo sepa.

Circuit Breaker — `client/lib/patterns/circuit_breaker/`

Protege las llamadas HTTP al backend. Tras 3 fallos consecutivos, el circuito se abre y la app devuelve datos en caché durante 30 segundos antes de intentar reconectar. (Ver sección 10 — Resiliencia.)

5. Pruebas automatizadas

5.1 TDD — Test-Driven Development

El desarrollo del backend siguió el ciclo **Red** → **Green** → **Refactor**.

Ejemplo aplicado al endpoint `/presupuesto/`:

Paso 1 — RED: Se escribió primero el test que fallaba porque el endpoint no existía:

```
def test_presupuesto_tiene_campos_requeridos(client):
    response = client.get("/presupuesto/")
    data = response.json()
    assert "presupuesto_total" in data
    assert "costo_total" in data
    assert "saldo_restante" in data
```

Paso 2 — GREEN: Se implementó el router mínimo para que el test pasara:

```
@router.get("/", response_model=PresupuestoResumen)
def obtener_presupuesto():
```

```
return _calcular_resumen()
```

Paso 3 — REFACTOR: Se extrajo `_calcular_resumen()` como función privada reutilizable y se añadió el cálculo de `porcentaje_utilizado`. Los tests existentes siguieron pasando sin modificación.

5.2 Pruebas unitarias y de mocks (PyTest)

Resultado: 30 tests — 100% passing — cobertura 82.5%

Módulo	Tests	Aspectos verificados
<code>test_auth.py</code>	6	Login, formato JWT, acceso con/sin token
<code>test_invitados.py</code>	8	CRUD completo, validación de datos, autenticación
<code>test_presupuesto.py</code>	6	Resumen, configuración, recálculo automático
<code>test_mock.py</code>	10	Mocking de bcrypt, store en memoria, JWT decode

El pipeline CI verifica `--cov-fail-under=80` en cada commit. Si la cobertura baja del 80%, el job falla y bloquea el merge.

----- coverage: platform darwin, python 3.11.14-final-0 -----				
Name	Stmts	Miss	Cover	Missing
auth/__init__.py	0	0	100%	
auth/dependencies.py	9	0	100%	
auth/jwt.py	17	0	100%	
data/__init__.py	0	0	100%	
data/store.py	7	0	100%	
main.py	20	4	80%	29-32
models/__init__.py	0	0	100%	
models/auth.py	7	0	100%	
models/invitado.py	20	0	100%	
models/presupuesto.py	13	0	100%	
routers/__init__.py	0	0	100%	
routers/auth.py	15	0	100%	
routers/invitados.py	34	5	85%	22-25, 42
routers/presupuesto.py	23	0	100%	
telemetry/__init__.py	0	0	100%	
telemetry/tracing.py	13	13	0%	5-25
tests/__init__.py	0	0	100%	
tests/conftest.py	24	0	100%	
tests/test_auth.py	24	0	100%	
tests/test_invitados.py	35	0	100%	
tests/test_mock.py	63	0	100%	
tests/test_presupuesto.py	25	0	100%	
tracing.py	11	11	0%	1-19

TOTAL	360	33	91%	

5.3 Pruebas de API — Newman (Postman)

La colección `tests/api/weddy.postman_collection.json` prueba todos los endpoints del sistema de forma secuencial:

1. Login → obtiene token JWT
2. GET /invitados/ → lista inicial vacía
3. POST /invitados/ con token → crea invitado → valida 201
4. PATCH /invitados/{id}/estado → cambia a "confirmado" → valida 200
5. GET /presupuesto/ → verifica que el costo se actualizó
6. DELETE /invitados/{id} con token → elimina → valida 204

Se ejecuta en CI con:

```
newman run tests/api/weddy.postman_collection.json \
  --reporters cli,htmlextra \
```

```
--reporter-html extra-export reports/newman-report.html
```

5.4 Pruebas de carga — k6

El script `tests/load/load_test.js` simula tráfico real sobre el backend con tres fases:

```
0s → 30s : ramp-up de 0 a 10 usuarios concurrentes
30s → 90s : carga sostenida con 10 usuarios
90s → 120s : ramp-down de 10 a 0 usuarios
```

Endpoints bajo prueba: `GET /health`, `GET /invitados/`, `GET /presupuesto/`.

5.5 Pruebas de GUI — Cypress

Los tests en `tests/gui/cypress/` prueban el flujo completo de la API desde una perspectiva de interfaz:

- Verifican que el backend responde en `/health`
- Validan el flujo de autenticación
- Comprueban que los endpoints devuelven la estructura JSON esperada

Se ejecutan en CI con el backend corriendo en background:

```
npx cypress run --headless
```

5.6 Reportes de pruebas

Todos los reportes se generan automáticamente en cada ejecución del CI y se publican como artefactos en GitHub Actions:

Herramienta	Artefacto	Formato
PyTest + Coverage	<code>pytest-reports</code>	HTML + XML
Allure	<code>allure-report</code>	HTML interactivo
Newman	<code>newman-report</code>	HTML
k6	<code>k6-results</code>	JSON
Cypress	<code>cypress-results</code>	JUnit XML + Screenshots
Semgrep	<code>semgrep-report</code>	JSON
Safety	<code>safety-report</code>	JSON

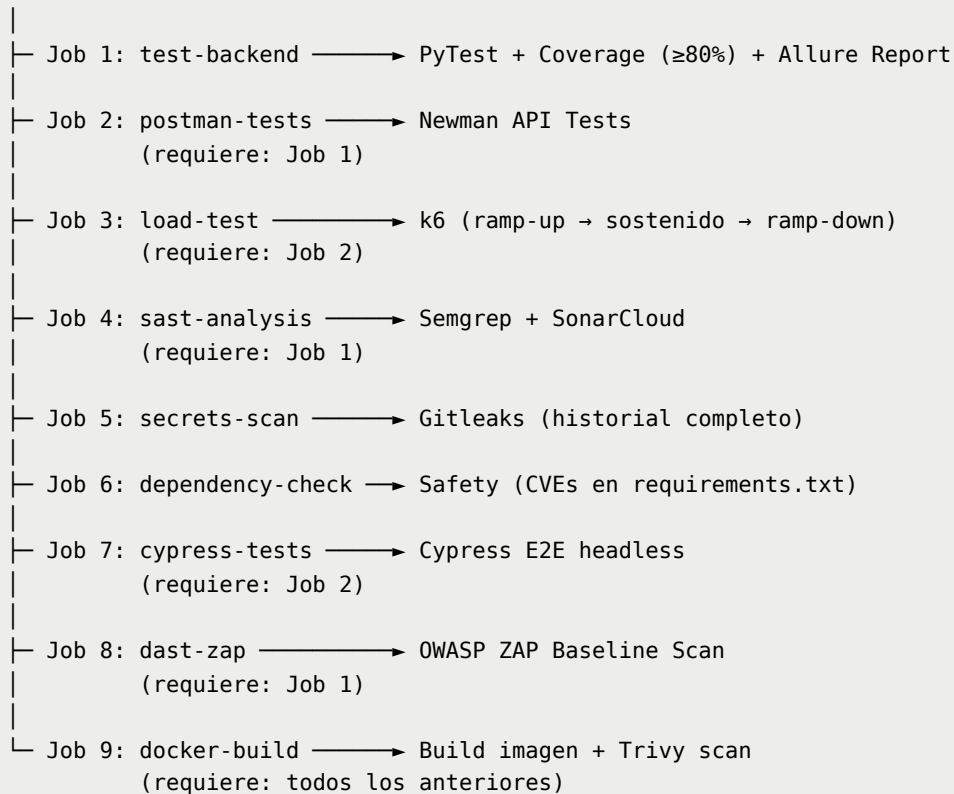
Herramienta	Artefacto	Formato
ZAP	<code>zap-report</code>	HTML + JSON
Trivy	<code>trivy-report</code>	SARIF

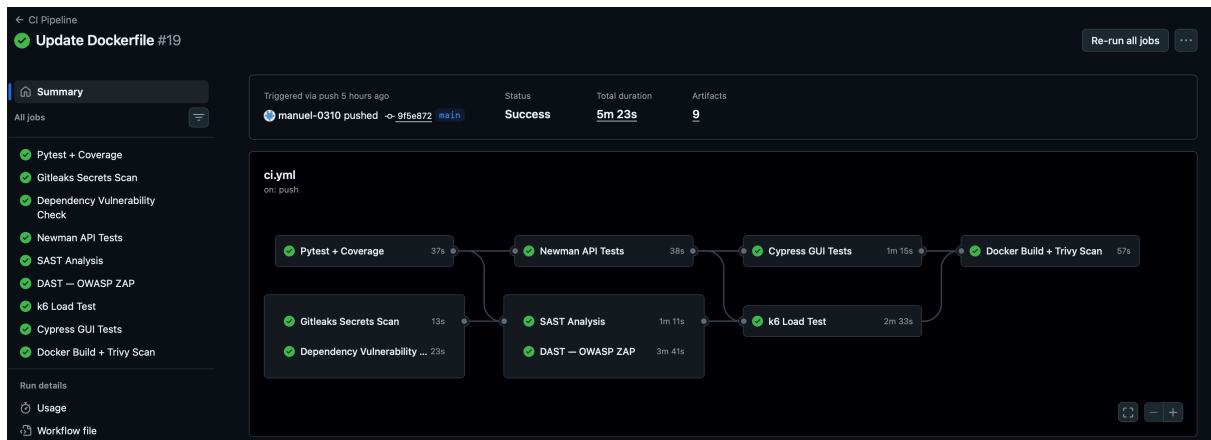
6. Pipeline CI/CD

6.1 Integración Continua — `.github/workflows/ci.yml`

Se activa en cada `push` o `pull_request` a `main`. Ejecuta 9 jobs en paralelo y en secuencia:

Commit → GitHub Actions





6.2 Despliegue Continuo — `.github/workflows/cd.yml`

Se activa automáticamente cuando el CI pasa en `main` (trigger: `workflow_run`):

CI ☒ en main

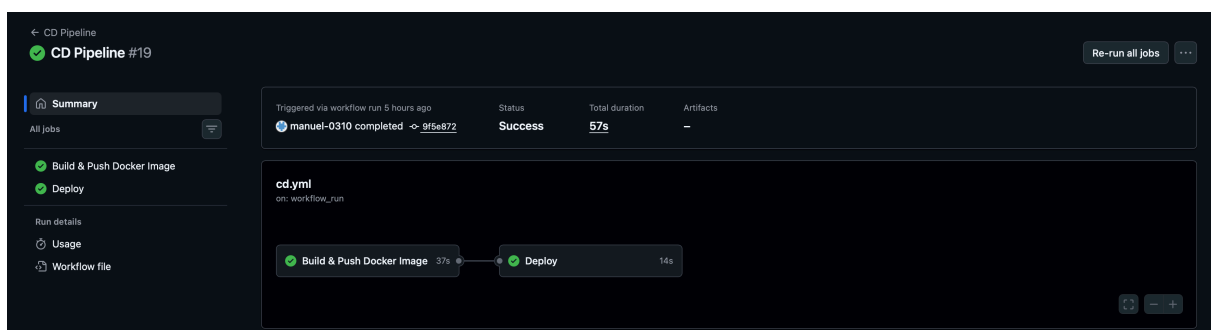
Job 1: Build & Push

- `docker build ./services`
- `docker push manuel0310/weddy-backend:latest`
- `docker push manuel0310/weddy-backend:{SHA}`

Job 2: Deploy

- Simula `kubectl set image`
- Crea comentario en el commit con la imagen publicada

La imagen queda disponible en DockerHub con dos tags: `latest` y el SHA del commit, permitiendo rollback a cualquier versión.



7. DevSecOps

7.1 Herramientas implementadas

Categoría	Herramienta	Job CI	Qué detecta
SAST	Semgrep	Job 4	Patrones inseguros en código Python
SAST	SonarCloud	Job 4	Calidad, bugs, code smells, cobertura
DAST	OWASP ZAP	Job 8	Vulnerabilidades en la API en ejecución real
Secrets Scanning	Gitleaks	Job 5	Secretos o credenciales en el historial git
Dependency Scanning	Safety	Job 6	CVEs conocidos en dependencias Python
Container Scanning	Trivy	Job 9	CVEs en la imagen Docker publicada

7.2 Seguridad en Kubernetes

Los manifiestos en `k8s/` implementan las mejores prácticas de seguridad de Kubernetes:

Práctica	Implementación	Archivo
RBAC	<code>ServiceAccount</code> con mínimo privilegio — sin acceso a secrets de otros namespaces	<code>k8s/rbac.yaml</code>
Secrets	<code>SECRET_KEY</code> y <code>JAEGGER_HOST</code> en <code>kind: Secret</code> — nunca en el código fuente	<code>k8s/secret.yaml</code>
Network Policy	Ingress solo desde namespace <code>monitoring</code> ; Egress restringido a DNS, Jaeger y HTTPS	<code>k8s/network-policy.yaml</code>
Pod Security	<code>runAsNonRoot: true</code> , <code>allowPrivilegeEscalation: false</code>	<code>k8s/deployment.yaml</code>
Capabilities	<code>drop: [ALL]</code> — el proceso no tiene ningún privilegio de kernel	<code>k8s/deployment.yaml</code>

7.3 Seguridad en el código

- **JWT HS256:** tokens firmados con clave secreta leída del entorno (`os.environ.get("SECRET_KEY")`), expiración de 3 horas
- **bcrypt:** contraseñas hasheadas con sal — la contraseña nunca se almacena en texto plano

- **Dependency injection:** `Depends(get_current_user)` en todos los endpoints de escritura
 - **Validación Pydantic:** todos los inputs validados automáticamente con tipos estrictos y `Field(gt=0)` donde aplica
 - **Usuario no-root en Docker:** el proceso uvicorn corre como `appuser` — si el contenedor es comprometido, no tiene privilegios de root
 - **CORS restringido:** `ALLOWED_ORIGINS` se configura por variable de entorno; los métodos y headers permitidos son mínimos
-

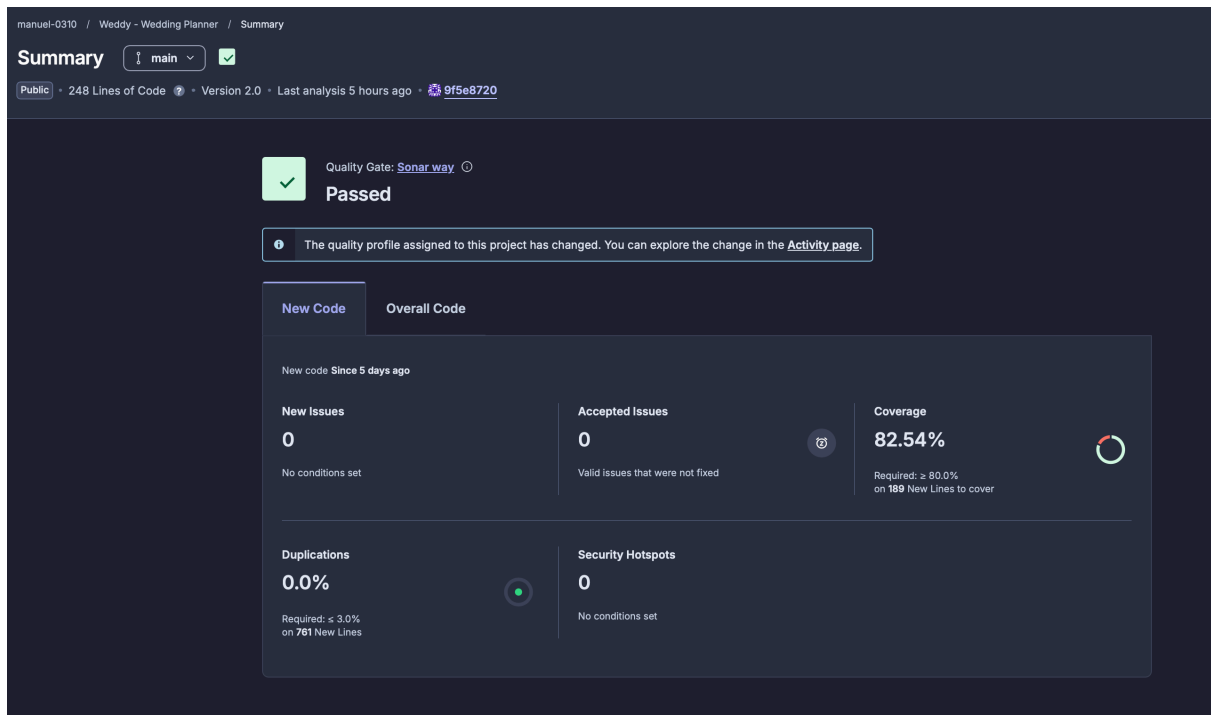
8. Análisis de calidad de código

8.1 SonarCloud — Quality Gate

SonarCloud analiza el backend en cada push a `main` como parte del Job 4 del CI.

Resultado final obtenido:

Métrica	Valor	Estado
Quality Gate	Passed	✓
Lines of Code	243	—
Cobertura	82.5%	✓ (umbral: 80%)
Duplicaciones	0.0%	✓
Issues abiertos	0	✓
Security Rating	A	✓
Reliability Rating	A	✓
Maintainability Rating	A	✓



8.2 Umbral de cobertura en CI

El pipeline verifica cobertura mínima en cada commit:

```
pytest tests/ --cov=. --cov-fail-under=80
```

Si la cobertura baja del 80%, el Job 1 falla, el Job 9 (docker-build) no se ejecuta y el CD no se dispara.

9. Observabilidad

9.1 Stack implementado

Herramienta	Puerto	Función
Prometheus	:9090	Recolección de métricas (scraping cada 10s)
Grafana	:3000	Dashboards de métricas + exploración de logs Loki
Jaeger	:16686	Trazabilidad distribuida por request
Loki	:3100	Centralización de logs de todos los contenedores
Promtail	—	Agente que envía logs Docker a Loki

9.2 Métricas recolectadas

La instrumentación automática de FastAPI expone en `/metrics`:

Métrica	Tipo	Descripción
<code>http_requests_total</code>	Counter	Total de requests por endpoint, método y código de respuesta
<code>http_request_duration_seconds</code>	Histogram	Latencia de cada endpoint
<code>http_request_size_bytes</code>	Summary	Tamaño de los requests entrantes
<code>http_response_size_bytes</code>	Summary	Tamaño de las respuestas enviadas

9.3 Flujo completo de observabilidad

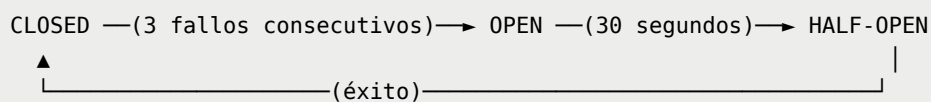


10. Resiliencia

10.1 Patrón implementado: Circuit Breaker + Fallback

Se implementó **Circuit Breaker con Fallback basado en caché** en la capa de datasources remotos del cliente Flutter.

Estados del circuito:



Configuración (`client/lib/patterns/circuit_breaker/circuit_breaker.dart`):

```
CircuitBreaker(
    name: 'invitados',
    failureThreshold: 3,                // abre tras 3 fallos c
    consecutiveFailures: 3,            // abre tras 3 fallos c
    resetTimeout: Duration(seconds: 30), // intenta reconecta
    r a los 30 segundos
)
```

Lógica de fallback:

```
Future<List<Invitado>> obtenerTodos() async {
    try {
        final data = await _cb.execute(() => ApiClient.get('/in
        vitados/'));
        _cache = _mapearLista(data); // actualiza caché en ca
        da éxito
        return _cache;
    } on CircuitBreakerException {
        if (_cache.isNotEmpty) return _cache; // fallback: da
        tos en caché
        rethrow;
    }
}
```

Cuando el circuito se abre, la app muestra un banner naranja con el mensaje *“Servicio no disponible. Mostrando datos en caché.”* — el usuario sigue viendo datos aunque el backend esté completamente caído.

10.2 Cobertura del patrón

Datasource	Endpoints protegidos
<code>InvitadoRemoteDatasource</code>	GET, POST /invitados/, PATCH y DELETE /invitados/{id}/estado
<code>PresupuestoRemoteDatasource</code>	GET /presupuesto/, POST /presupuesto/configurar

Cada datasource tiene su propio CircuitBreaker independiente: un fallo en invitados no afecta al circuito de presupuesto.

10.3 Comportamientos observados

Escenario	Estado del circuito	Comportamiento
Backend activo	CLOSED	Datos frescos del backend
1 o 2 fallos	CLOSED	Error puntual, sigue intentando
3 fallos consecutivos	OPEN	Banner naranja + datos en caché
Backend caído + circuito abierto	OPEN	App usable con datos cacheados
30 segundos después	HALF-OPEN	Una petición de prueba
Backend vuelve en HALF-OPEN	CLOSED	Reconecta, banner desaparece

11. Decisiones de seguridad y escalabilidad

11.1 Decisiones de seguridad

JWT HS256 sobre sesiones con cookie:

JWT permite autenticación stateless — el backend no almacena estado de sesión, lo que facilita escalar horizontalmente sin compartir sesiones entre instancias. La firma HS256 garantiza integridad sin requerir infraestructura PKI.

bcrypt para contraseñas:

bcrypt aplica un factor de costo (salt rounds) que hace los ataques de fuerza bruta computacionalmente inviables incluso con hardware moderno. La contraseña nunca se almacena en texto plano.

CORS restringido por variable de entorno:

En lugar del wildcard `*`, los orígenes permitidos se configuran con `ALLOWED_ORIGINS`. En desarrollo se usa `http://localhost`; en producción se especificarían los dominios reales del frontend.

Secretos en Kubernetes Secrets, no en código:

El `SECRET_KEY` vive en `k8s/secret.yaml` como objeto `kind: Secret` y se inyecta al pod como variable de entorno. El código lee `os.environ.get("SECRET_KEY")` — nunca hay un secreto hardcodeado en el repositorio.

Usuario no-root en Docker:

El Dockerfile crea un usuario sin privilegios (`appuser`) y el proceso uvicorn corre bajo ese usuario. Si el contenedor es comprometido, el atacante no obtiene acceso root al host.

Network Policy en Kubernetes:

Solo los pods del namespace `monitoring` pueden hacer peticiones al backend. Cualquier otro tráfico de ingress es denegado por defecto. Esto limita el radio de explosión en caso de compromiso de otro servicio.

11.2 Decisiones de escalabilidad

Backend stateless:

El backend no guarda estado de sesión ni de autenticación — toda la información de identidad viaja en el JWT. Esto permite añadir réplicas sin coordinación entre instancias.

Kubernetes Deployment con 2 réplicas:

El `k8s/deployment.yaml` especifica `replicas: 2`. Kubernetes distribuye el tráfico entre ambas réplicas y reinicia automáticamente pods que fallen (liveness probe sobre `/health`).

FastAPI con Uvicorn ASGI:

FastAPI sobre Uvicorn maneja I/O de forma asíncrona, soportando muchas conexiones concurrentes con un solo proceso. Para producción se escalaría con múltiples workers (`uvicorn --workers 4`).

Docker como unidad de despliegue:

Cada commit a `main` que pase el CI genera una imagen Docker etiquetada con el SHA del commit. Esto permite hacer rollback a cualquier versión anterior con un solo comando.

Datos en memoria como decisión consciente:

Para el alcance académico, el store en memoria es suficiente. En producción se reemplazaría por PostgreSQL con conexión pooling — la arquitectura del backend (routers → data store) facilita esta sustitución sin cambiar la lógica de negocio.

12. Retos técnicos y soluciones

SonarCloud seguía marcando issues de seguridad después de mover la SECRET_KEY a env vars

Problema: Se movió `SECRET_KEY` de un valor hardcodeado a `os.environ.get("SECRET_KEY", "fallback")`. SonarCloud seguía marcando issues en

`routers/auth.py` porque la contraseña del usuario `admin` aún era un literal `b"weddy2024"` usado directamente en `bcrypt.hashpw()`.

Solución: Aplicar el mismo patrón de env vars para la contraseña del usuario demo:

```
_ADMIN_PASSWORD = os.environ.get("ADMIN_PASSWORD", "weddy2024").encode()
_USERS: dict[str, bytes] = {
    "admin": bcrypt.hashpw(_ADMIN_PASSWORD, bcrypt.gensalt()),
}
```

Resultado: El literal `"weddy2024"` ya no aparece directamente como argumento de una función de hashing — SonarCloud dejó de flaggearlo. La contraseña de desarrollo sigue siendo la misma, pero configurable por entorno.